Лабораторная работа 4 Параллельное умножение матриц

Виноградов Дима Сергеевич · группа 11-400

Реализованы ijk_seq, ijk_omp, ikj_seq, ikj_omp и blocked_omp на C++17. Матрицы double хранятся динамически, входные данные воспроизводимы; параллельные алгоритмы распределяют независимые строки или блоки C. Система: Windows, Intel Core i5-12450H, 8 ядер / 12 логических процессоров, GCC 15.2.0; сборка -O3 -std=c++17 -fopenmp.

Методика: один прогрев, три измерения и медиана; omp_get_wtime() измеряет только умножение, без выделения памяти, заполнения, обнуления C и проверки. После каждого запуска проверено $|C - C_{эт}| \le 10^{-9} + 10^{-9}|C_{эт}|$ относительно ijk_seq; нарушений нет. Нулевой отсчёт указан как < 0,001 с из-за наблюдаемого шага таймера около 1 мс.

1 Влияние размера матрицы

Для параллельных версий p = 8; блок b = 32; все времена в секундах.

N	ijk_seq	ijk_omp	ikj_seq	ikj_omp	blocked_omp
128	0,001	0,001	< 0,001	< 0,001	< 0,001
256	0,010	0,003	0,003	0,001	0,002
512	0,330	0,045	0,027	0,006	0,017
1024	6,841	0,719	0,345	0,055	0,145

2 Влияние числа потоков

N = 512, b = 32; базы Tijk = 0,330 с и Tikj = 0,027 с. S = Tseq / Tomp, E = S/p; E приведена в процентах. Точка 12 добавлена к ТЗ.

p	ijk_omp с	S ijk	E ijk %	ikj_omp с	S ikj	E ikj %	blocked_omp с
1	0,340	0,97	97,1	0,026	1,04	103,8	0,078
2	0,157	2,10	105,1	0,014	1,93	96,4	0,038
4	0,079	4,18	104,4	0,008	3,37	84,4	0,031
8	0,045	7,33	91,7	0,006	4,50	56,2	0,017
12	0,046	7,17	59,8	0,005	5,40	45,0	0,014
16	0,040	8,25	51,6	0,007	3,86	24,1	0,018

3 Влияние размера блока

N = 512, p = 8; отношение включает многопоточность и блочную обработку.

Блок b	Время blocked_omp, с	Tikj_seq / Tblocked_omp
16	0,019	1,42
32	0,017	1,59
64	0,019	1,42

Вывод

При N = 1024 смена порядка ijk на ikj сократила последовательное время с 6,841 до 0,345 с, примерно в 19,8 раза. В ikj внутренний цикл последовательно обращается к B и C, улучшая локальность и создавая более удобные условия для автовекторизации. Самой быстрой при N = 1024 оказалась ikj_omp на 8 потоках: 0,055 с. В серии N = 512 лучшее время ikj_omp получено на 12 потоках: 0,005 с, S = 5,40 и E = 45 %, а на 16 потоках время выросло до 0,007 с. Для ijk_omp рост после 8 потоков замедлился, но на 16 достигнуто S = 8,25, поэтому полного плато во всех алгоритмах нет. Эффективность ijk немного выше 100 % при 2 и 4 потоках может объясняться различиями в использовании кешей, частоте и разбросом измерений; это не гарантия сверхлинейного масштабирования. Блок 32 дал минимум 0,017 с среди 16, 32 и 64, однако blocked_omp уступила ikj_omp: выигрыш от повторного использования данных здесь не компенсировал накладные расходы блочных циклов. Масштабирование ограничивают ресурсы ядер, обмен с памятью, синхронизация и планирование потоков; 16 программных потоков делят 12 логических процессоров. На малых матрицах накладные расходы и разрешение таймера мешают надёжно сравнивать очень короткие вычисления.